

Perbandingan Algoritma *Dynamic programming*, *Divide and Conquer*, dan Moore dalam Minimasi Deterministic Finite Automaton untuk Mengelompokkan *State* Ekuivalen

Jordan Tenggara - 24060124120044
Departemen Informatika
Universitas Diponegoro
Semarang, Indonesia
jordantenggara@undip.ac.id

Abstract—Deterministic Finite Automaton (DFA) dapat memiliki beberapa state yang berbeda secara penamaan, tetapi memiliki perilaku yang sama terhadap seluruh kemungkinan input. Kondisi tersebut membuat DFA dapat disederhanakan melalui proses minimasi dengan mengelompokkan state-state ekuivalen. Penelitian ini membandingkan tiga algoritma, yaitu Moore, Dynamic Programming, dan Divide and Conquer, dalam minimasi DFA. Dataset yang digunakan terdiri atas 16 DFA dalam format JSON dengan variasi jumlah state, accepting state, dan struktur transisi. Setiap DFA diuji menggunakan ketiga algoritma, kemudian hasilnya dibandingkan berdasarkan jumlah state hasil minimasi, rasio reduksi state, kesesuaian terhadap Moore, dan waktu eksekusi. Hasil eksperimen menunjukkan bahwa ketiga algoritma menghasilkan jumlah state hasil minimasi yang sama pada seluruh dataset. Dari sisi waktu eksekusi, Moore memperoleh rata-rata waktu terendah sebesar 0.000114 detik, diikuti Divide and Conquer sebesar 0.000260 detik, dan Dynamic Programming sebesar 0.000497 detik. Dengan demikian, perbedaan utama antaralgoritma tidak terletak pada hasil minimasi, tetapi pada strategi pemrosesan dan efisiensi waktu eksekusi.

Keywords—*Deterministic Finite Automaton*, minimasi DFA, state ekuivalen, Moore, Dynamic Programming, Divide and Conquer

I. PENDAHULUAN

Finite Automata (FA) merupakan sebuah model komputasi fundamental yang mempunyai peran krusial dalam bidang teori bahasa dan otomata, khususnya teori ilmu komputer dan aplikasi praktis numerik, dan banyak bidang lain [4]. *Deterministic Finite Automata* (DFA), di sisi lain merupakan salah satu contoh dari FA yang memberikan solusi paling sederhana dalam merepresentasikan dan menalarakan terkait sistem terstruktur, pola, aturan, dan proses [4]. DFA juga merupakan salah satu dari formalisme komputasional yang paling sederhana [3]. Oleh karena itu, DFA merupakan hal yang penting untuk dipelajari, baik cara DFA bekerja maupun bagaimana kita bisa merepresentasikan hal dengan DFA secara efisien.

DFA diterapkan dengan beberapa state yang berbeda secara penamaan, namun berperilaku sama terhadap seluruh kemungkinan input. *State-state* tersebut disebut sebagai *state* ekuivalen karena tidak dapat dibedakan berdasarkan bahasa yang diterima selama proses transisi. Untuk mengoptimalkan sebuah DFA, dapat dilakukan sebuah teknik minimasi DFA yang bertujuan untuk membuat automata yang mencapai hasil yang sama, namun dengan jumlah *state* yang lebih sedikit [1]. Jadi, minimasi DFA tidak mengubah fungsi atau tujuan dari suatu DFA, hanya menyederhanakan DFA terkait.

Teknik minimasi DFA yang umum digunakan, salah satunya adalah dengan menggunakan *partition refinement*, yaitu tiap *state* akan disempurnakan hingga menyisakan *state* ekuivalen yang bersifat stabil. Algoritma Moore merupakan algoritma minimasi DFA yang menggunakan konsep *partition refinement*, dengan konsep memisahkan *final state* dan *non-final state* yang kemudian akan diperhalus partisinya [1]. Algoritma ini dapat dijadikan sebagai standar dalam menganalisa *approach-approach* lain dalam minimasi DFA [2].

Algoritma-algoritma lain dapat dieksplorasi untuk mencari *approach* untuk meminimalkan sebuah DFA untuk menghasilkan hasil yang sama, namun dengan struktur berbeda. Pada penelitian ini, algoritma *Dynamic programming* serta *Divide and conquer* akan digunakan untuk meminimasi DFA. Algoritma Moore, sebagai algoritma standar yang sering digunakan dalam minimasi DFA, akan digunakan sebagai faktor pembanding dengan membandingkan jumlah *state* hasil minimasi, rasio reduksi *state*, dan waktu eksekusi masing-masing algoritma.

II. BATASAN MASALAH

Penulisan makalah tentunya terdapat beberapa batasan untuk mendefinisikan ruang lingkup penelitian, sehingga penelitian dapat berjalan dengan lancar. Batasan yang pertama dalam penelitian adalah semua proses minimasi yang dilakukan oleh tiap-tiap algoritma, tidak dapat menerima input gambar DFA, melainkan menerima input DFA yang sudah dikonversikan ke bentuk JSON.

Batasan lainnya adalah metrik penilaian yang digunakan dalam penelitian hanya mencantumkan *state* ekuivalen, jumlah *state* hasil minimasi, dan performa masing-masing algoritma berdasarkan waktu komputasi dan eksekusi dalam meminimasi DFA.

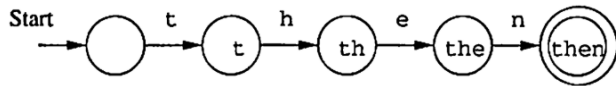
III. DASAR TEORI

A. Finite Automata

Finite Automaton (FA) merupakan sebuah model yang mempunyai banyak kegunaan penting dalam berbagai perangkat lunak maupun perangkat keras. Beberapa contoh dari kegunaan FA adalah perangkat lunak untuk mendesain dan mengecek peralihan dari sirkuit *digital*, *lexical analyzer* yang mengkompilasi input teks ke bentuk unit logikal, perangkat lunak yang digunakan untuk *scanning* berbagai teks dalam bentuk yang besar, dan perangkat lunak untuk memverifikasi tipe-tipe sistem yang memiliki jumlah *state* yang terbatas [5].

Salah satu contoh dari Finite Automaton adalah sebuah model automata untuk merekognisi kata “then”.

Fig. 1. Contoh modeling dari finite automaton



Pada gambar di atas, terdapat 5 state dengan gap antar state diisi dengan input di mana setiap state akan mengalami progresi dalam isinya yaitu penambahan string dari input, dimulai dari state awal yang berwujud state kosong dan diakhiri oleh state yang berlingkaran dua, yaitu finish state.

Automata merupakan konsep yang sangat penting dalam bidang penelitian batasan komputasi. Beberapa konsep penting adalah penelitian terkait *decidability*, merupakan penentuan permasalahan-permasalahan yang dapat dipecahkan oleh komputer dan *intractability*, bidang studi di mana solusi dari pemecahan masalah memiliki kompleksitas yang sama apabila besaran dari input bertambah [5].

B. Deterministic Finite Automata

Deterministic Finite Automaton (DFA) merupakan bagian dari *Finite Automaton* di mana sebuah FA diberikan konsep deterministik, yaitu tiap state pada automata hanya dapat menerima satu input untuk transisi ke state selanjutnya [5]. DFA sendiri mempunyai beberapa syarat agar dapat dikatakan sebagai *deterministic finite automaton*, yaitu:

- 1) Sebuah himpunan state yang terdapat, biasanya disimbolkan dengan Q .
- 2) Sebuah himpunan simbol masukan, biasanya disimbolkan dengan Σ .
- 3) Automata memiliki fungsi transisi yang menerima simbol input dan mengembalikan state berikutnya.
- 4) Sebuah state awal
- 5) Sebuah himpunan state akhir, biasanya di simbolkan dengan F .

Dari berbagai syarat agar sebuah automata dapat dikatakan sebagai *deterministic finite automata*, DFA dapat direpresentasikan dengan himpunan yang berisi lima komponen, yaitu:

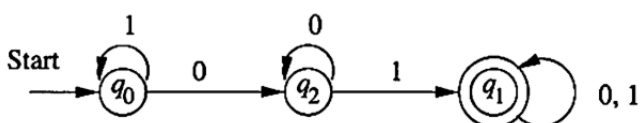
$$A = (Q, \Sigma, \delta, q_0, F)$$

Dimana A merupakan nama dari DFA, Q merupakan himpunan state, Σ merupakan input, δ merupakan fungsi transisi, q_0 merupakan state awal, dan F merupakan himpunan state finish.

DFA dapat digambarkan dengan dua cara, yaitu dengan *transition diagram*, merupakan penggambaran berbentuk graf dan *transition table*, merupakan penggambaran berbentuk senarai bersifat tabular yang berisikan fungsi transisi, input, dan kondisi state [5].

Contoh dari transition diagram antara lain:

Fig. 2. Contoh Transition Diagram



Automata di atas merupakan contoh dari DFA yang menerima semua input string dengan substring 01. Bentuk lain dari gambar diagram transisi adalah tabel transisi, yaitu sebagai berikut.

TABLE I. CONTOH TABEL TRANSISI

	0	1
$\rightarrow q_0$	q_2	q_0
$* q_1$	q_1	q_1
q_2	q_2	q_1

C. State Ekuivalen dan Minimasi DFA

State Ekuivalen mempunyai makna, yaitu apabila terdapat dua state yang tidak dapat dibedakan dalam segi input. Sehingga, untuk setiap string yang diberikan, masing-masing state dapat menghasilkan output yang sama, sehingga state-state tersebut yang bersifat ekuivalen dapat digabungkan menjadi satu dikarenakan tidak akan mengubah hasil selain dari perubahan struktur DFA [1].

Oleh karena itu, minimasi DFA dapat diartikan sebagai sebuah proses untuk membentuk atau merekonstruksi sebuah DFA kebentuk yang lebih sederhana atau simpel, tanpa merubah atau memengaruhi output yang semestinya dikeluarkan dengan input yang sama seperti sebelumnya [1]. Rekonstruksi ini dapat dicapai dengan mengidentifikasi tiap-tiap state yang mempunyai sifat ekuivalen, lalu menggabungkan menjadi satu, hingga menghasilkan himpunan state yang sudah terminimasi.

D. Algoritma Moore

Algoritma Moore merupakan salah satu algoritma yang lazim dipakai dalam minimasi DFA. Algoritma ini bekerja dengan menggunakan pendekatan bernama *partition refinement*, yaitu sebuah proses yang dimulai dengan pembagian state menjadi dua kelompok, yaitu final dan non-final state. Setelah dibagi, tiap-tiap partisi akan diperhalus berdasarkan pola dan tujuan transisinya. Di sisi lain, apabila terdapat pola yang berbeda, kelompok tersebut akan dipartisi kembali dan dilakukan penghalusan hingga tidak ada pola yang berbeda sehingga tidak akan dilakukan partisi lagi. Hasil akhir dari partisi yang stabil dapat dikatakan merepresentasikan kelompok state yang ekuivalen [1].

Pemilihan algoritma ini dikarenakan algoritma Moore sudah banyak dikaji karena mempunyai kompleksitas yang relevan dalam proses minimasi DFA [2]. Sehingga akan dijadikan sebagai standar dalam melakukan komparasi dengan algoritma-algoritma lain.

E. Algoritma Divide and conquer

Algoritma *Divide and conquer* merupakan sebuah metode atau strategi kuat dalam mendesain algoritma efisien untuk kasus yang bersifat asimtotik, atau kasus di mana terdapat nilai yang terus mendekati batas seiring berjalannya waktu.

Penyelesaian yang digunakan oleh algoritma *divide and conquer* adalah penyelesaian yang bersifat rekursif, pemanggilan terus menerus. Apabila permasalahan yang diambil merupakan permasalahan kecil, maka dapat langsung dijadikan sebagai basis untuk pemecahan masalah. Namun, apabila permasalahan yang didapatkan cukup besar, maka

penanganan rekursif akan diambil, dengan cara sebagai berikut [6].

- 1) Divide, proses membagi masalah menjadi satu atau lebih submasalah.
- 2) Conquer, penyelesaian masing-masing submasalah dengan cara rekursif.
- 3) Combine, menyatukan berbagai penyelesaian yang sudah dibuat hingga menjadi sebuah solusi untuk permasalahan awal.

Jadi, algoritma *divide and conquer* merupakan sebuah algoritma yang akan memecah sebuah permasalahan utama menjadi submasalah yang lebih kecil, lalu memecahkan tiap-tiap submasalah, dan menggabungkan semua solusi dari submasalah hingga menciptakan sebuah solusi untuk menjawab permasalahan awal, di mana tiap penyelesaian dilakukan secara rekursif [6].

Penerapan metode *divide and conquer* akan diterapkan dengan cara membagi setiap proses pengelompokkan *state* menjadi beberapa bagian yang lebih kecil. Maka dari itu, setiap kelompok akan diproses dan dicari hingga ditemukan *state* ekuivalennya. Setelah setiap kelompok ditemukan masing-masing *state* ekuivalennya, maka akan digabungkan menjadi DFA baru yang sudah dilakukan minimasi.

F. Dynamic programming melalui Table-Filling Method

Dynamic programming atau program dinamis, mempunyai beberapa konsep yang *similiar* dengan *divide and conquer*, yaitu membagi sebuah masalah menjadi beberapa submasalah dan menciptakan solusi untuk masing-masing solusi, lalu menggabungkannya menjadi satu solusi utama. Akan tetapi, *dynamic programming* menerapkan konsep tersebut apabila terdapat permasalahan yang bersifat *overlapping* atau tumpang tindih, sehingga solusi sebuah submasalah yang didapatkan dapat digunakan untuk menyelesaikan submasalah lain [6]. Namun, berbeda dengan *divide and conquer* yang akan mencari solusi dari tiap-tiap submasalah, *dynamic programming* akan menyimpan solusi dari submasalah tersebut sehingga dapat digunakan kembali tanpa harus menyelesaikan permasalahan kembali [6].

Untuk membuat sebuah algoritma *dynamic programming*, terdapat sebuah proses yang berisikan empat langkah [6], yaitu.

- 1) Characterize, solusi optimal akan dikarakterisasi strukturnya.
- 2) Recursively, nilai dari solusi paling optimal akan didefinisikan secara rekursif.
- 3) Compute, nilai dari solusi paling optimal akan dihitung dengan pola biasanya bottom-up.
- 4) Construct, hasil komputasi akan digunakan untuk membangun sebuah solusi yang paling optimal.

Pada penelitian ini, algoritma *Dynamic programming* akan dilakukan melalui pendekatan metode *Table-Filling* berbasis memoization. Konsep *dynamic programming* akan digunakan dengan cara memandang tiap-tiap *state* sebagai submasalah dan menghitung solusi dari permasalahan yang mana solusi terkait akan disimpan. Tiap-tiap *state* akan dianalisa dan diperiksa berdasarkan fungsi transisinya, di mana apabila sebuah *state* mempunyai fungsi transisi yang sudah diketahui dan sama, maka akan digabung dengan anggapan bahwa *state-state* tersebut merupakan *state* ekuivalen. Sebaliknya, apabila

transisi suatu *state* menuju *state* lain belum diketahui, maka *state* akan dianggap berbeda dan dicari solusi baru.

IV. METODOLOGI

A. Desain Penelitian

Pendekatan yang digunakan dalam penelitian adalah pendekatan eksperimen komputasional, yaitu eksperimen untuk membandingkan performa dari tiga algoritma dalam meminimasi sebuah DFA, yaitu *Dynamic programming*, *Divide and conquer*, dan Moore. Masing-masing algoritma akan mendapatkan perlakuan yang sama dalam meminimasi DFA, baik itu dataset DFA, resource, maupun rangkaian proses yang akan dilakukan sehingga akan menghasilkan hasil yang konsisten.

Rangkaian proses yang akan dilalui oleh masing-masing algoritma antara lain pembacaan dataset DFA, validasi struktur DFA, penerapan algoritma, pencatatan hasil penerapan algoritma, dan pengukuran performa algoritma.

Alur penelitian terdiri atas beberapa tahapan, mulai dari mempersiapkan dataset DFA dengan format JSON, membaca dan memvalidasi masing-masing DFA, menjalan ketiga algoritma, lalu membandingkan hasil dari masing-masing algoritma berdasarkan jumlah *state* setelah dilakukan minimasi, reduksi *state*, dan perbandingan terhadap algoritma standar, yaitu Moore, serta waktu eksekusi masing-masing algoritma.

Hasil dari eksperimen komputasi akan disajikan dalam bentuk tabel dan grafik sehingga mempermudah proses analisis perbandingan masing-masing performa algoritma.

B. Representasi dan Validasi Data DFA

Dataset DFA direpresentasikan dengan format JSON. Setiap DFA dalam dataset mengikuti format struktur dari DFA itu sendiri, mulai dari nama DFA, daftar *state*, alphabet input, *state* awal, *state* final, dan fungsi transisi.

Secara umum, struktur data DFA yang digunakan memuat komponen berikut:

- 1) name, yaitu nama atau identitas DFA
- 2) states, yaitu daftar seluruh *state* dalam DFA
- 3) alphabet, yaitu daftar simbol input yang digunakan
- 4) start_state, yaitu *state* awal
- 5) accept_states, yaitu daftar *state* final
- 6) transitions, yaitu fungsi transisi dalam bentuk pasangan *state* dan simbol input menuju *state* tujuan.

Agar masing-masing algoritma dapat dijalankan, setiap DFA dalam dataset akan divalidasi terlebih dahulu. Validasi dataset dilakukan agar setiap DFA dalam dataset mengandung semua struktur DFA yang sudah dijelaskan diatas. Tahap validasi ini dilakukan agar data yang diproses oleh masing-masing algoritma merupakan data DFA yang lengkap dan bersifat deterministik.

C. Penerapan Algoritma Moore

Algoritma Moore merupakan algoritma standar yang dipilih sebagai pembanding utama dikarenakan Moore meminimasi DFA dengan konsep *partition refinement* serta algoritma Moore sudah lazim digunakan dalam proses minimasi DFA.

Implementasi Moore adalah pembentukan partisi awal, yaitu final *state* dan non-final *state*. Setelah dilakukan partisi, tiap-tiap partisi akan dianalisis berdasarkan pola transisi dan akan ditentukan apakah diperlukan untuk partisi kembali apabila terdapat tujuan transisi yang berbeda.

Selanjutnya adalah tahap *refinement*, yaitu tahap penghalusan tiap partisi hingga masing-masing partisi bersifat stabil. Apabila kondisi setiap partisi sudah stabil, maka tiap-tiap kelompok *state* akan dianggap sebagai kelompok *state* ekuivalen di mana jumlah kelompok *state* akan dihitung sebagai jumlah *state* hasil minimasi.

Langkah umum Algoritma Moore dalam penelitian ini adalah sebagai berikut:

```

Input : DFA
Output : Kelompok state ekuivalen

1) Bentuk partisi awal final state
   dan non-final state.
2) Untuk setiap state, hitung
   signature berdasarkan tujuan
   transisinya.
3) Pecah kelompok jika terdapat
   state dengan signature berbeda.
4) Ulangi proses refinement hingga
   partisi tidak berubah.
5) Kembalikan partisi akhir sebagai
   kelompok state ekuivalen.

```

Penerapan algoritma Moore dalam *pseudo-code*:

Algorithm 1 Moore Minimization

Input: DFA $M = (Q, \Sigma, \delta, q_0, F)$

Output: Partition P containing equivalent state groups

```

1:  $P \leftarrow \{F, Q - F\}$ 
2: repeat
3:    $P_{old} \leftarrow P$ 
4:    $P \leftarrow \emptyset$ 
5:   for each group  $G$  in  $P_{old}$  do
6:     Buckets  $\leftarrow \emptyset$ 
7:     for each state  $q$  in  $G$  do
8:       signature  $\leftarrow$  empty tuple
9:       for each symbol  $a$  in  $\Sigma$  do
10:        target  $\leftarrow \delta(q, a)$ 
11:        index  $\leftarrow$  group number of target in
            $P_{old}$ 
12:        append index to signature
13:      end for
14:      insert  $q$  into Buckets[signature]
15:    end for
16:    for each bucket  $B$  in Buckets do

```

```

17:      add  $B$  to  $P$ 
18:    end for
19:  end for
20: until  $P = P_{old}$ 
21: return  $P$ 

```

D. Penerapan Algoritma Dynamic programming melalui Table-Filling

Algoritma *Dynamic programming* akan diimplementasikan melalui pendekatan table-filling berbasis memoization, yaitu setiap pasangan *state* akan diperlakukan sebagai submasalah di mana hasil dari pemeriksaan submasalah tersebut akan disimpan dalam sebuah tabel memo. Melalui tabel memo, hasil pemeriksaan dapat digunakan kembali apabila ditemukan hasil pemeriksaan pasangan *state* selanjutnya mempunyai kesamaan dengan hasil yang sudah disimpan dalam tabel memo.

Tahapan *Dynamic programming* dimulai dengan mengkategorikan setiap pasangan *state*, di mana pasangan yang terdiri atas satu *state* final dan satu *state* non-final akan ditandai berbeda. Pasangan *state* lain akan diperiksa dan disimpan datanya dalam tabel memo, apabila ditemukan perbedaan maka akan dilakukan penandaan dan ditambahkan ke dalam tabel memo. Pasangan *state* yang tidak ditandai maka akan dianggap sebagai *state* ekuivalen dan digabungkan ke dalam kelompok yang sama.

Langkah umum *Dynamic programming* melalui Table-Filling dalam penelitian ini adalah sebagai berikut:

```

Input : DFA
Output : Kelompok state ekuivalen

1) Bentuk seluruh pasangan state.
2) Inisialisasi tabel memo untuk
   menyimpan status pasangan state.
3) Tandai pasangan final dan non-
   final sebagai berbeda.
4) Periksa pasangan lain berdasarkan
   transisi masing-masing state.
5) Jika transisi menuju pasangan
   yang berbeda, tandai pasangan
   saat ini sebagai berbeda.
6) Ulangi proses hingga tidak ada
   perubahan pada tabel memo.
7) Gabungkan pasangan yang tidak
   berbeda sebagai state ekuivalen.

```

Penerapan algoritma *dynamic programming* dalam *pseudo-code*:

Algorithm 2 Dynamic programming Table-Filling

Input: DFA $M = (Q, \Sigma, \delta, q_0, F)$

Output: Partition P containing equivalent state groups

```

1: Memo  $\leftarrow$  empty table

```

```

2:   for each pair of states (p, q),  $p \neq q$  do
3:       if  $p \in F$  and  $q \notin F$ , or  $p \notin F$  and  $q \in F$  then
4:           Memo[p, q]  $\leftarrow$  different
5:       else
6:           Memo[p, q]  $\leftarrow$  unknown
7:       end if
8:   end for
9:   changed  $\leftarrow$  true
10:  while changed = true do
11:      changed  $\leftarrow$  false
12:      for each pair of states (p, q),  $p \neq q$  do
13:          if Memo[p, q] = unknown then
14:              for each symbol a in  $\Sigma$  do
15:                   $p' \leftarrow \delta(p, a)$ 
16:                   $q' \leftarrow \delta(q, a)$ 
17:                  if Memo[p', q'] = different then
18:                      Memo[p, q]  $\leftarrow$  different
19:                      changed  $\leftarrow$  true
20:                      break
21:                  end if
22:              end for
23:          end if
24:      end for
25:  end while
26:  P  $\leftarrow$  empty partition
27:  for each pair of states (p, q),  $p \neq q$  do
28:      if Memo[p, q] = unknown then
29:          merge p and q into the same group
30:      end if
31:  end for
32:  return P

```

E. Penerapan Algoritma Divide and conquer

Minimasi DFA melalui algoritma *Divide and conquer* bersifat eksperimental dalam hal pengelompokkan *state* ekuivalen. Himpunan keseluruhan *state* akan dibagi menjadi dua subhimpunan yang mana akan terjadi pembagian kembali secara rekursif. Pembagian *state* akan dilakukan hingga ditentukan bahwa *state* sudah cukup kecil, lalu dilakukan pemeriksaan apabila *state* bersifat ekuivalen dengan *state* lain dalam satu subhimpunannya.

Hasil dari masing-masing subhimpunan akan digabungkan, dengan penggabungan tiap sub himpunan akan dilakukan pemeriksaan kembali apakah masih terdapat *state* ekuivalen di subhimpunan yang berbeda. Bentuk akhir dari

penggabungan adalah kelompok *state* ekuivalen yang merupakan hasil DFA yang sudah diminimasi.

Langkah umum *Divide and conquer* dalam penelitian ini adalah sebagai berikut:

Input : DFA dan himpunan *state*

Output : Kelompok *state* ekuivalen

- 1) Jika jumlah *state* dalam subhimpunan kecil, lakukan pemeriksaan ekuivalensi langsung.
- 2) Jika belum, bagi himpunan *state* menjadi dua subhimpunan.
- 3) Proses subhimpunan kiri secara rekursif.
- 4) Proses subhimpunan kanan secara rekursif.
- 5) Gabungkan hasil kedua subhimpunan.
- 6) Periksa ekuivalensi lintas-subhimpunan pada tahap combine.
- 7) Kembalikan hasil akhir sebagai kelompok *state* ekuivalen.

Penerapan algoritma *divide and conquer* dalam pseudo-code:

Algorithm 3 Divide-and-Conquer State Equivalence Grouping

Input: DFA $M = (Q, \Sigma, \delta, q_0, F)$, state set $S \subseteq Q$

Output: Partition P containing equivalent state groups

- 1: if $|S| \leq \text{threshold}$ then
- 2: $P \leftarrow$ group states in S by direct equivalence checking
- 3: return P
- 4: end if
- 5: Split S into S_{left} and S_{right}
- 6: $P_{\text{left}} \leftarrow \text{DivideConquerGrouping}(M, S_{\text{left}})$
- 7: $P_{\text{right}} \leftarrow \text{DivideConquerGrouping}(M, S_{\text{right}})$
- 8: $P \leftarrow$ merge P_{left} and P_{right}
- 9: for each group G_1 in P_{left} do
- 10: for each group G_2 in P_{right} do
- 11: $r_1 \leftarrow$ representative state of G_1
- 12: $r_2 \leftarrow$ representative state of G_2
- 13: if r_1 and r_2 are equivalent then
- 14: merge G_1 and G_2 in P

15:	end if
16:	end for
17:	end for
18:	return P

F. Metrik Evaluasi

Setelah masing-masing algoritma dijalankan dan didapatkan hasil minimasi DFA-nya, akan dilakukan evaluasi dengan beberapa metrik. Metrik pertama adalah jumlah *state* awal, yaitu jumlah *state* DFA sebelum dilakukan minimasi. Metrik kedua merupakan kebalikan dari metrik pertama, yaitu jumlah *state* akhir setelah dilakukan minimasi. Metrik ketiga adalah jumlah kelompok *state* ekuivalen, yaitu hasil pengelompokan *state* berdasarkan pola transisinya.

Metrik-metrik diatas juga akan dilakukan perhitungan jumlah dan persentase reduksi *state* untuk mempermudah penglihatan terhadap hasil dari penyederhanaan DFA. Di sisi lain, performa waktu komputasi juga akan diukur untuk membandingkan efisiensi masing-masing algoritma. Algoritma Moore juga akan digunakan sebagai fokus kesesuaian apakah hasil dari algoritma *Dynamic programming* serta *Divide and conquer* dapat menyamai hasil dari algoritma Moore.

Secara ringkas, metrik evaluasi yang digunakan adalah sebagai berikut:

- 1) jumlah *state* awal
- 2) jumlah *state* hasil minimisasi
- 3) jumlah kelompok *state* ekuivalen
- 4) jumlah reduksi *state*
- 5) persentase reduksi *state*
- 6) waktu eksekusi rata-rata
- 7) waktu eksekusi minimum dan maksimum
- 8) kesesuaian hasil terhadap Moore

V. HASIL DAN PEMBAHASAN

A. Hasil Eksperimen

Tiga algoritma minimasi DFA dijalankan, yaitu Moore, *Dynamic programming*, dan *Divide and conquer*, pada dataset DFA dalam bentuk JSON. Dataset yang digunakan terdiri atas 16 DFA dengan variasi jumlah *state*, accepting *state*, dan struktur transisi. Setiap DFA diproses menggunakan tiga algoritma, sehingga diperoleh total 48 hasil pengujian. Metrik yang digunakan meliputi jumlah *state* awal, jumlah *state* hasil minimisasi, persentase reduksi *state*, waktu eksekusi rata-rata, serta kesesuaian hasil terhadap algoritma Moore.

Setelah menerima hasil eksperimen ketiga algoritma, ditemukan bahwa hasil minimasi tiap-tiap DFA merupakan sama dari segi jumlah *statenya*. Persamaan jumlah *state* ini menunjukkan bahwa algoritma *Dynamic programming* dan *Divide and conquer* berhasil untuk menghasilkan kelompok *state* ekuivalen yang sama dengan algoritma Moore, algoritma standar yang sering digunakan dalam minimasi DFA. Perbedaan hanya terletak pada waktu eksekusi, yang akan ditunjukkan pada tabel perbandingan sebagai berikut.

TABLE II. RINGKASAN HASIL EKSPERIMEN PER ALGORITMA

Algoritma	Rata-rata State Hasil	Rata-rata Reduksi State	Rata-rata Waktu Eksekusi	Konsistensi dengan Moore
Moore	3.875	51.37%	0.000114s	Ya
<i>Divide and conquer</i>	3.875	51.37%	0.000260s	Ya
DP	3.875	51.37%	0.000497s	Ya

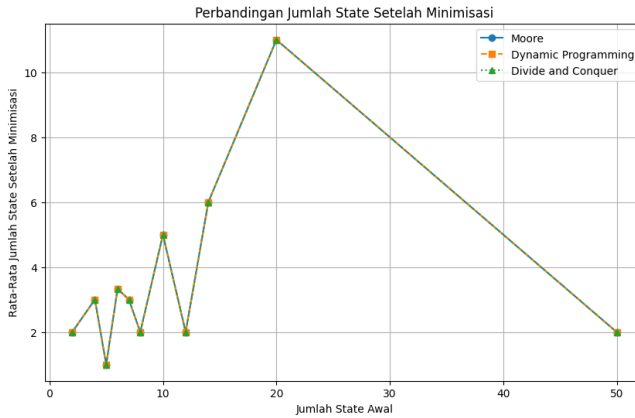
Metrik lain berupa reduksi *state* juga ditemukan bahwa masing-masing jenis DFA mengalami tingkat reduksi *state* yang berbeda-beda. DFA yang teridentifikasi mengandung banyak *state* ekuivalen mengalami banyak reduksi, sedangkan DFA yang sudah minimal (sedikit mengandung *state* ekuivalen) mengalami sedikit reduksi atau bahkan tidak mengalami reduksi sama sekali. Hasil reduksi *state* beberapa DFA dapat dilihat pada tabel berikut.

TABLE III. CONTOH HASIL REDUKSI STATE PADA BEBERAPA DFA

DFA	State Awal	State Hasil	Reduksi State
DFA_02_Even_Number_of_Zeroes	2	2	0%
DFA_03_All_NonFinal_Equivalent	5	1	80%
DFA_11_Three_Class_12_State	12	2	83.33%
DFA_14_Large_Two_Block_20_State	20	2	90%
DFA_15_Long_Chain_20_State	20	20	0%
DFA_16_Large_Two_Block_50_State	50	2	96%

Berikut merupakan gambar yang berupa grafik di mana ditunjukkan bahwa masing-masing algoritma mendapatkan jumlah *state* hasil minimasi yang sama untuk masing-masing DFA.

Fig. 3. Perbandingan Jumlah State Setelah Minimasi

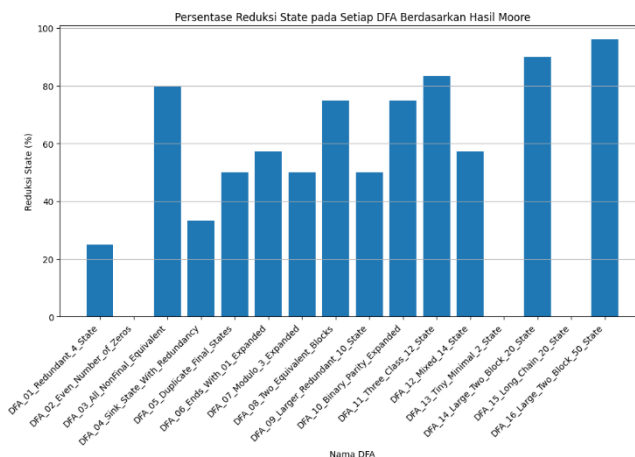


B. Analisis Jumlah State Hasil Minimasi

DFA dengan struktur yang memiliki banyak *state* ekuivalen mengalami reduksi *state* yang besar. DFA_16_Large_Two_Block_50_State mengalami reduksi dari 50 *state* menjadi 2 *state*, sehingga persentase reduksinya mencapai 96%. DFA_14_Large_Two_Block_20_State juga mengalami reduksi besar dari 20 *state* menjadi 2 *state*, dengan persentase reduksi 90%. Hasil ini menunjukkan bahwa minimisasi DFA sangat efektif pada automata yang memiliki banyak *state* dengan sifat ekuivalen.

Di sisi lain, DFA yang sudah minimal atau memiliki sedikit *state* ekuivalen mengalami sedikit atau bahkan tidak mengalami reduksi *state*. DFA_02_Even_Number_of_Zeros tetap memiliki 2 *state* setelah minimisasi, sedangkan DFA_15_Long_Chain_20_State tetap memiliki 20 *state*. Hal ini menunjukkan bahwa jumlah *state* yang besar tidak selalu berarti DFA dapat direduksi secara signifikan. Reduksi hanya terjadi apabila terdapat *state-state* yang ekuivalen dan dapat digabung tanpa mengubah bahasa yang diterima oleh DFA.

Fig. 4. Persentasi Reduksi State Setiap DFA

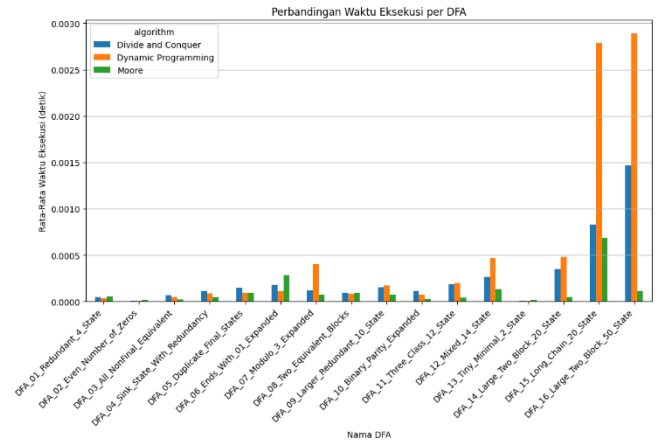


C. Analisis Waktu Eksekusi

Berdasarkan waktu eksekusi, algoritma Moore memperoleh rata-rata waktu terendah, yaitu 0.000114 detik, disusul oleh algoritma *Divide and conquer* dengan rata-rata waktu 0.00026 detik, terakhir adalah *Dynamic programming* memiliki rata-rata waktu tertinggi, sebesar 0.000497 detik.

Oleh karena itu, dapat disimpulkan bahwa algoritma Moore merupakan algoritma paling cepat dari kecepatan komputasi.

Fig. 5. Perbandingan Waktu Eksekusi DFA



Setelah dianalisis, algoritma Moore cenderung lebih efisien dikarenakan menggunakan konsep *partition refinement*, yaitu memperhalus kelompok *state* secara langsung hingga mencapai partisi stabil. Pendekatan ini tidak memerlukan pemeriksaan seluruh pasangan *state* seperti *Dynamic programming*. *Dynamic programming* memerlukan pemeriksaan pasangan *state*, sehingga waktu eksekusinya dipengaruhi oleh banyaknya kombinasi pasangan *state* yang harus diproses. Ketika jumlah *state* bertambah, jumlah pasangan *state* juga meningkat secara signifikan, sehingga waktu eksekusi *Dynamic programming* dapat meningkat lebih besar.

Divide and conquer menunjukkan hasil yang lebih baik dibanding *Dynamic programming* secara rata-rata pada dataset terbaru. Hal ini terlihat terutama pada DFA berukuran lebih besar. Pada DFA_14_Large_Two_Block_20_State, *Divide and conquer* membutuhkan waktu 0.00026 detik, sedangkan *Dynamic programming* membutuhkan 0.000497 detik. Pada DFA_15_Long_Chain_20_State, *Divide and conquer* membutuhkan 0.000832 detik, sedangkan *Dynamic programming* membutuhkan 0.002783 detik. Pada DFA_16_Large_Two_Block_50_State, *Divide and conquer* membutuhkan 0.001465 detik, sedangkan *Dynamic programming* membutuhkan 0.002888 detik. Hasil ini menunjukkan bahwa strategi pembagian *state* dapat mengurangi beban pemeriksaan pasangan *state* pada kasus tertentu, terutama ketika ukuran DFA lebih besar.

Walaupun *divide and conquer* lebih unggul dari *dynamic programming* dalam segi waktu eksekusi, algoritma Moore tetapkan algoritma yang tercepat. Hal ini disebabkan oleh adanya tahap *combine* pada *divide and conquer*, yaitu proses penggabungan kembali hasil dari subhimpunan kelompok *state* dan pemeriksaan ekuivalensi antar subhimpunan. Dengan demikian, *Divide and conquer* dapat menjadi lebih cepat dibanding *Dynamic programming* pada DFA yang lebih besar, tetapi Moore tetap menjadi pendekatan paling stabil pada dataset yang digunakan.

D. Analisis Konsistensi

Moore digunakan sebagai standar karena merupakan algoritma minimisasi DFA berbasis *partition refinement*. Pada hasil eksperimen, seluruh nilai kesesuaian terhadap Moore bernilai benar. Artinya, hasil *Dynamic programming* dan

Divide and conquer konsisten dengan hasil Moore pada seluruh dataset yang diuji.

Hal ini menunjukkan bahwa kedua pendekatan alternatif dapat digunakan untuk mengelompokkan *state* ekuivalen pada data uji yang digunakan. *Dynamic programming* menghasilkan kelompok *state* ekuivalen melalui pemeriksaan pasangan *state* berbasis memoization, sedangkan *Divide and conquer* menghasilkan kelompok *state* melalui pembagian himpunan *state* dan penggabungan hasil antar subhimpunan. Walaupun proses komputasinya berbeda, keduanya menghasilkan hasil akhir yang sama dengan Moore.

E. Temuan

Berdasarkan hasil eksperimen, Moore menjadi pendekatan yang paling stabil dari sisi waktu eksekusi rata-rata. Hal ini sesuai dengan karakter Moore yang secara langsung melakukan refinement terhadap partisi *state*. Moore juga tetap efisien pada DFA berukuran besar yang memiliki banyak *state* ekuivalen, seperti DFA_16_Large_Two_Block_50_State.

Dynamic programming memiliki kelebihan dari sisi kejelasan proses pemeriksaan pasangan *state* karena setiap pasangan dapat ditandai sebagai berbeda atau tidak berbeda. Pada DFA kecil, *Dynamic programming* dapat memiliki waktu yang cepat dikarenakan jumlah pasangan *state* yang diperiksa masih sedikit. Namun, pada DFA yang lebih besar, pendekatan ini cenderung lebih lambat karena jumlah pasangan *state* bertambah secara eksponensial.

Divide and conquer menunjukkan peningkatan posisi dibandingkan *Dynamic programming* pada dataset terbaru. Pendekatan ini lebih cepat pada DFA berukuran besar karena proses pengelompokan dilakukan secara bertahap melalui pembagian himpunan *state*. Namun, pendekatan ini tetap memiliki kelemahan pada tahap combine, karena *state* ekuivalen dapat berasal dari subhimpunan yang berbeda sehingga tetap diperlukan pemeriksaan lintas-subhimpunan.

Secara umum, hasil penelitian menunjukkan bahwa ketiga algoritma mampu menghasilkan kelompok *state* ekuivalen yang sama pada dataset uji. Perbedaan utama terletak pada karakteristik proses dan waktu eksekusi. Moore lebih unggul secara rata-rata, *Dynamic programming* cocok untuk menjelaskan proses pemeriksaan pasangan *state*, sedangkan *Divide and conquer* menjadi alternatif yang lebih kompetitif dibanding *Dynamic programming* pada DFA berukuran lebih besar. Namun, hasil ini masih dibatasi oleh ukuran dataset, representasi input dalam bentuk JSON, serta lingkungan eksekusi Google Colab yang dapat memengaruhi pengukuran waktu. Oleh karena itu, hasil waktu eksekusi hanya dapat dijadikan perbandingan relatif pada lingkungan eksperimen yang sama, bukan sebagai hasil mutlak untuk seluruh jenis DFA.

VI. KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil eksperimen yang telah dilakukan, algoritma Moore, *Dynamic programming*, dan *Divide and conquer* dapat digunakan untuk mengelompokkan *state* ekuivalen pada Deterministic Finite Automaton (DFA). Seluruh algoritma menghasilkan jumlah *state* hasil minimisasi yang sama pada dataset yang diuji, sehingga *Dynamic programming* dan *Divide and conquer* menunjukkan hasil yang konsisten terhadap Moore sebagai algoritma pembandingan utama. Hasil minimisasi juga menunjukkan

bahwa DFA dengan banyak *state* ekuivalen dapat mengalami reduksi *state* yang besar, seperti pada DFA dengan struktur blok ekuivalen, sedangkan DFA yang sudah minimal atau memiliki banyak *state* yang dapat dibedakan tidak mengalami pengurangan jumlah *state*.

Dari sisi waktu eksekusi, Moore menjadi algoritma dengan rata-rata waktu eksekusi paling rendah pada dataset yang digunakan. *Divide and conquer* berada pada posisi kedua dan menunjukkan performa yang lebih baik dibandingkan *Dynamic programming* pada beberapa DFA berukuran lebih besar. *Dynamic programming* tetap menghasilkan hasil minimisasi yang benar, tetapi waktu eksekusinya cenderung lebih tinggi karena proses pemeriksaan pasangan *state* bertambah seiring meningkatnya jumlah *state*. Dengan demikian, perbedaan utama antara ketiga pendekatan tidak terletak pada hasil minimisasi, tetapi pada strategi pemrosesan dan efisiensi waktu eksekusi. Hasil penelitian ini tetap dipengaruhi oleh batasan dataset, format input JSON, serta lingkungan eksekusi Google Colab yang dapat memengaruhi hasil pengukuran waktu.

B. Saran

Untuk pengembangan penelitian selanjutnya, dataset DFA dapat diperluas dengan jumlah *state*, alphabet, dan variasi struktur transisi yang lebih beragam agar analisis performa menjadi lebih komprehensif. Penelitian lanjutan juga dapat menambahkan algoritma minimisasi DFA lain seperti Hopcroft atau Brzozowski sebagai pembandingan, sehingga hasil eksperimen tidak hanya terbatas pada Moore, *Dynamic programming*, dan *Divide and conquer*. Selain itu, metrik evaluasi dapat diperluas dengan mengukur penggunaan memori, jumlah iterasi, dan jumlah pasangan *state* yang diperiksa. Pengembangan berikutnya juga dapat diarahkan pada pembentukan DFA hasil minimisasi secara eksplisit dan visualisasi diagram automata agar hasil pengelompokan *state* ekuivalen lebih mudah dianalisis.

LAMPIRAN

- 1) Link kode program:
<https://colab.research.google.com/drive/1L4a1d3BWK-JhASoAlgnvcxvg6Zok9O5A?usp=sharing>
- 2) Link Dataset:
https://drive.google.com/file/d/11XDLHJZwh-gO2B4MqVe2S4AYG0BHT_KQ/view?usp=sharing
- 3) Link Hasil Eksperimen:
<https://drive.google.com/file/d/1QCFMm9gldTznNHdWi8zA2gYdxUXs6dJn/view?usp=sharing>

REFERENCES

- [1] J. Berstel, L. Boasson, O. Carton, and I. Fagnot, "Minimization of automata," arXiv:1010.5318, 2010. [Online]. Available: https://drive.google.com/file/d/1P8vvXXVcfsK62A66NHwXxzf7fM1I3/view?usp=drive_link
- [2] F. Bassino, J. David, and C. Nicaud, "On the average complexity of Moore's *state* minimization algorithm," in Proc. 26th Int. Symp. Theoretical Aspects of Computer Science (STACS 2009), Freiburg, Germany, 2009, pp. 123–134. [Online]. Available: https://drive.google.com/file/d/15dEi3ico0Ps_92lqL2SyWxLXaQEjKrYf/view?usp=drive_link
- [3] J. Martens and A. Wijs, "An evaluation of massively parallel algorithms for DFA minimization," Electron. Proc. Theor. Comput. Sci., vol. 409, pp. 138–153, 2024, doi: 10.4204/EPTCS.409.13. [Online]. Available: https://drive.google.com/file/d/1nmjHAMpaB4hLgy_325mzVN5MtM_uq_fLA/view?usp=drive_link
- [4] J. Michaliszyn and J. Otop, "Minimization of deterministic finite automata modulo the edit distance," in Proc. 50th Int. Symp. Mathematical Foundations of Computer Science (MFCS 2025),

- LIPIcs, vol. 345, Art. no. 77, pp. 77:1–77:17, 2025, doi: 10.4230/LIPIcs.MFCS.2025.77. [Online]. Available: <https://drive.google.com/file/d/17zBnt2z0ZjzORY5mEXIYb9TOYf7TTVG3/view?usp=sharing>
- [5] J. E. Hopcroft, R. Motwani, and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 2nd ed. Boston, MA, USA: Addison-Wesley, 2001. [Online]. Available: https://drive.google.com/file/d/1muW3kQlhxseAZ629nvr4J_IVMpiFy4S/view?usp=sharing
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009. [Online]. Available: <https://drive.google.com/file/d/1f-kKOPuVA1YDxWC7p2fTxjbeamBGqKIJ/view?usp=sharing>

PERNYATAAN

Dengan ini saya menyatakan bahwa makalah yang saya susun merupakan hasil karya saya sendiri dan tidak merupakan saduran, terjemahan, maupun plagiasi dari karya orang lain. Seluruh sumber yang digunakan telah dicantumkan secara benar sesuai dengan kaidah penulisan ilmiah.

Terkait penggunaan teknologi kecerdasan buatan (Artificial Intelligence), saya menyatakan bahwa (hapus salah satu):

☐ Saya **menggunakan** alat bantu AI sebagai pendukung dalam penyusunan makalah ini dengan rincian sebagai berikut:

- Nama alat AI: ChatGPT
- Tujuan penggunaan: membantu dalam menyesuaikan gaya bahasa penulisan laporan, pemahaman konsep dan rancangan implementasi algoritma dan program.
- Ruang lingkup penggunaan: penjelasan konsep dan algoritma, penyesuaian isi laporan, pseudocode, dan kode eksperimen yang tetap dilakukan pemeriksaan dan perubahan oleh penulis.

Penggunaan AI, apabila ada, hanya bersifat sebagai alat bantu dan tidak menggantikan peran saya sebagai penulis utama. Seluruh isi, analisis, serta kesimpulan dalam makalah ini tetap merupakan tanggung jawab saya sepenuhnya

Semarang, 7 Juni 2026



Jordan Tenggara
24060124120044